

APOSTILA DO ALUNO — AULA 3

Modelagem de Sistemas, Ferramentas e Introdução à UML

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 11/09/2026 (sexta-feira) | Horário: 07:30 às 11:30 (4h — ritmo bem acelerado)

Objetivos desta aula

- Definir modelagem de sistemas e entender por que ela é feita.
- Diferenciar modelos estruturais de comportamentais.
- Ter uma visão geral da UML e dos diagramas que veremos na disciplina.
- Reconhecer, em prévia, os elementos do Diagrama de Casos de Uso (ator, caso de uso, fronteira).
- Conhecer ferramentas de modelagem, com foco no draw.io.

Retomada das Aulas 1 e 2

Este conteúdo estava previsto para o fim da Aula 2, mas não coube no tempo — por isso abre a Aula 3, fechando o ciclo iniciado naquela aula antes de começarmos a representar tudo em UML.

Relembrando as definições

Antes de entrar em Critérios de Aceite, um resumo rápido do que já vimos nas Aulas 1 e 2:

- Regra de Negócio: uma condição que o negócio impõe, independente de sistema — ex.: “pedido mínimo de R\$ 15 para entrega”.
- Requisito Funcional (RF): o que o sistema deve fazer — uma ação ou funcionalidade concreta.
- Requisito Não-Funcional (RNF): como o sistema deve se comportar — desempenho, segurança, usabilidade.
- A cadeia do ciclo: regra de negócio → requisito (RF/RNF) → critério de aceite → teste.

Critérios de Aceite

Um critério de aceite é uma condição objetiva que diz quando um Requisito Funcional pode ser considerado “pronto e correto”. Ele fecha o ciclo regra de negócio → requisito → critério de aceite → teste: sem ele, “pronto” é opinião; com ele, vira algo que dá para testar.

Formato simples: Dado / Quando / Então

Dado [um contexto inicial], Quando [uma ação acontece], Então [o sistema deve reagir assim]. Esse formato é conhecido como Gherkin, usado depois em testes de aceitação.

Exemplo, para o RF “o sistema deve bloquear o envio do pedido para a cozinha até que o pagamento seja confirmado”:

- Dado um pedido com pagamento pendente, Quando o atendente tentar enviar o pedido à cozinha, Então o sistema deve impedir o envio e exibir uma mensagem de pagamento pendente.

Mais um exemplo: tente sozinho

RF: “o sistema deve liberar a entrega apenas para endereços em até 5 km da loja” (Regra 5 do TechBurger, Aula 1). Antes de olhar a resposta, tentem escrever o Dado/Quando/Então.

- Resposta: Dado um endereço de entrega informado no pedido, Quando o sistema verificar a distância até a loja, Então: se estiver a até 5 km, libera a entrega; se estiver fora do raio, oferece só a opção de retirada no balcão.

Pratique

Peguem 1 Requisito Funcional que o grupo escreveu na Aula 2 e escrevam o critério de aceite dele, no formato Dado/Quando/Então (consultem a Apostila da Aula 2, seção 8.2, se precisarem lembrar).

Mais um exemplo de Critério de Aceite

RF do TechBurger: “se o item selecionado estiver sem estoque, o sistema sugere um substituto” (Regra 6).

Dado um item sem estoque selecionado no pedido, Quando o cliente tentar adicioná-lo ao carrinho, Então o sistema deve impedir a adição e sugerir um item substituto do mesmo tipo.

Reparem: o “Então” tem duas partes (impede + sugere) — as duas precisam estar no critério, senão ele fica incompleto.

Erros comuns ao escrever critérios de aceite

- Critério vago — “o sistema deve funcionar direito” não dá pra testar.
- Critério que só repete o RF, sem detalhar o resultado esperado.
- Critério que testa duas coisas ao mesmo tempo, sem deixar claro qual delas falhou.
- Esquecer o “Dado” — sem contexto inicial, não dá pra saber quando o critério se aplica.
- Regra prática: se dois colegas testarem o mesmo critério e chegarem a resultados diferentes, ele não está bom o suficiente.

1. Modelagem de Sistemas — Definição

Modelagem de sistemas é o processo de representar, por meio de modelos (diagramas, esquemas), o funcionamento de um sistema antes ou durante seu desenvolvimento. Um modelo é sempre uma simplificação da realidade: ele mostra o que importa para o propósito do momento, sem tentar representar tudo.

Assim como um engenheiro civil faz uma planta antes de construir, o desenvolvedor faz um modelo antes de programar — corrigir um erro no modelo é muito mais barato do que corrigir código já escrito.

1.1 Por que corrigir um modelo é mais barato que corrigir código

Quanto mais cedo um erro de entendimento é encontrado, mais barato é corrigi-lo. Um erro identificado ainda no levantamento de requisitos custa pouco: basta reescrever uma frase. O mesmo erro, se só for percebido depois que o sistema já está em produção, pode exigir reescrever código, retestar tudo e ainda lidar com clientes insatisfeitos. Esse é o principal motivo para modelar antes de programar.

Fase em que o erro é encontrado	Custo relativo de corrigir
Levantamento de requisitos	1x
Modelagem / projeto	5x
Codificação	10x
Produção (sistema já no ar)	100x ou mais

Esses números são uma estimativa clássica da Engenharia de Software (baseada nos estudos de Barry Boehm), não um valor exato — mas a tendência de crescimento vale para praticamente qualquer projeto.

Exemplo concreto: a Regra 4 do TechBurger (desconto PIX) tinha uma ambiguidade — “aplica automaticamente” não dizia se era antes ou depois de confirmar o pedido. Pega isso ainda no levantamento de requisitos: custo 1x, só ajustar o texto. Só percebe na codificação: custo 10x. Só aparece com o sistema no ar, cliente reclamando de desconto duplicado: custo 100x ou mais.

Mais um exemplo, agora com a Biblioteca: a regra “Ler Mais” (um associado só pode pegar um livro novo com menos de 3 emprestados) tem uma ambiguidade real — “emprestados” conta os livros atrasados também, ou só os dentro do prazo? Pega essa dúvida no levantamento de requisitos: custo 1x, só perguntar e documentar a resposta. Só percebe na codificação: custo 10x. Só aparece com o sistema no ar, associado reclamando que não conseguiu pegar um livro que deveria poder pegar: custo 100x+.

1.2 Breve histórico da UML

Até o início dos anos 1990, cada especialista tinha seu próprio jeito de desenhar sistemas orientados a objetos — havia dezenas de notações concorrentes e incompatíveis entre si. Em meados da década, três métodos se destacavam: o de Grady Booch, o OMT de James Rumbaugh e o OOSE de Ivar Jacobson. Os três autores se juntaram para unificar suas notações em uma única linguagem — daí o nome Unified Modeling Language. Em 1997 a UML foi padronizada pela OMG (Object Management Group), organização internacional responsável por mantê-la até hoje. A versão usada atualmente é a UML 2.x.

Debate: E se não tivéssemos UML?

Imagem: cada empresa, cada time, cada dev com sua própria notação de diagrama. Um dev sai do time — o próximo não entende os diagramas que ele deixou. Uma consultoria externa entra no projeto — precisa aprender a notação do zero. Isso é literalmente o cenário pré-1997, antes da unificação.

Debate rápido: que outros problemas um “cada um com seu jeito” causaria num time de verdade?

1.3 UML no mercado: só o necessário

Mesmo em times ágeis, modelar não desapareceu — só ficou mais leve (“Agile Modeling”): documentar o suficiente pra o time entender, nada além disso.

Um diagrama não é um entregável pro cliente, é uma ferramenta de comunicação pro time — pode ser um rascunho no quadro, não precisa ser “obra de arte”.

- Regra prática: modele o suficiente pra tirar uma dúvida ou alinhar o time — não modele por modelar.

1.3.1 UML no mercado: exemplos reais

- Vagas de Analista de Sistemas e Arquiteto de Software costumam pedir “leitura de diagramas UML” como requisito.
- Ferramentas como Jira, Confluence e Azure DevOps permitem anexar diagramas UML direto nas histórias e épicos.
- Times que praticam Domain-Driven Design (DDD) desenhavam Diagramas de Classes pra alinhar o modelo de domínio antes de codificar.
- Mesmo sem “UML formal”, ferramentas como Miro e Excalidraw reaproveitam a mesma notação — o padrão sobrevive, só muda o formato do arquivo.

1.3.2 Quem usa modelagem no dia a dia

- Analista de Requisitos — desenha Casos de Uso pra alinhar escopo com o cliente.
- Arquiteto de Software — usa Diagrama de Componentes e Implantação pra decidir a estrutura do sistema.
- Product Owner — lê (não desenha) diagramas pra entender o que o time está construindo.
- QA / Testador — usa Diagrama de Atividades e critérios de aceite pra planejar casos de teste.
- Nem todo papel desenha diagrama — mas quase todo papel técnico precisa ler um.

1.4 UML 1.x → 2.x: o que mudou

- UML 1.x (1997-2003): focada em sistemas orientados a objetos “clássicos”, com poucos diagramas comportamentais.
- UML 2.x (a partir de 2005): adiciona o Diagrama de Estrutura Composta, o de Comunicação, o de Temporização e o de Visão Geral de Interação.
- UML 2.x também reforça o Diagrama de Componentes, pensando em sistemas maiores e distribuídos.
- Hoje praticamente só se usa a 2.x — é a versão que vamos usar nesta disciplina.

1.5 Debate: Quanto modelar é suficiente?

- Cenário A — MVP de startup: app de delivery pra validar em 1 semana, com um time de 2 desenvolvedores.
- Cenário B — Sistema interno de RH: vai rodar por anos, usado por várias áreas da empresa, time médio.
- Cenário C — Sistema bancário: transferências entre contas, sujeito a auditoria de compliance, time de 15+ desenvolvedores.
- Cenário D — Sistema de saúde: prontuário eletrônico com dados sensíveis (LGPD), integração com outros hospitais e laboratórios — aqui o risco de um erro não é só financeiro, pode afetar a saúde de alguém.

Para cada cenário, debatam: vocês modelariam tudo em UML antes de codificar, ou só o essencial? Quais diagramas (se algum) vocês fariam?

- Regra prática: quanto maior o risco de um erro não percebido, mais vale a pena modelar antes de codificar.

2. Tipos de modelo

2.1 Modelos estruturais

Representam a estrutura estática do sistema — o que existe: classes, objetos, componentes.

2.2 Modelos comportamentais

Representam o comportamento dinâmico do sistema — o que acontece: casos de uso, atividades, sequência de interações.

2.3 Características de um bom modelo

- Abstração — mostra só o que importa para o objetivo do modelo.
- Clareza — qualquer stakeholder consegue entender, mesmo sem ser técnico.
- Consistência — não se contradiz com outros modelos do mesmo sistema.
- Nível de detalhe adequado ao público que vai usar o modelo.

Como reconhecer um modelo ruim

- Diagrama de Classes com 40 atributos numa única classe — ninguém consegue ler de relance.
- Nomes genéricos: classe “Coisa1”, atributo “dado2” — não comunicam nada sobre o sistema real.
- Relações sem multiplicidade: não dá pra saber se é “1 pra 1” ou “1 pra muitos”.
- Se vocês olharem um diagrama e não conseguirem explicar o que ele representa em 10 segundos, ele provavelmente viola alguma das características da seção anterior.

2.4 Estrutural x Comportamental: um exemplo lado a lado

O mesmo pedido do TechBurger, visto pelos dois ângulos: como estrutura, ele envolve um Pedido, vários Itens e um Cliente (as “coisas” que existem). Como comportamento, ele é uma sequência: o cliente escolhe os itens, confirma o pagamento, e o pedido vai para a cozinha (o que acontece, em ordem).

Outro par de olhares, agora com login no sistema: como estrutura, ele envolve um Usuário, uma Sessão e uma Permissão — um Usuário tem um Perfil de acesso (as “coisas” envolvidas). Como comportamento, é uma sequência: o usuário digita as credenciais, o sistema valida, e uma sessão é criada (a sequência de tentativas até o acesso ser liberado ou negado).

Mais um par de olhares, com a Biblioteca “Ler Mais”: como estrutura, ela envolve um Aluno, um Livro e um Empréstimo — um Aluno pode ter vários Empréstimos ativos. Como comportamento, é uma sequência: o aluno pede um livro, o sistema verifica o limite de 3, e confirma ou recusa o empréstimo.

2.5 Por que a UML tem tantos diagramas diferentes?

Pense na planta de um prédio: a planta baixa mostra os cômodos e paredes, a planta elétrica mostra os fios e tomadas, e a planta hidráulica mostra os canos — todas descrevem o mesmo prédio, mas cada uma foca em um aspecto diferente, para um público diferente (o pedreiro usa uma, o eletricista usa outra). A UML funciona do mesmo jeito: cada diagrama é uma “planta” diferente do mesmo sistema, e nenhum diagrama sozinho conta a história inteira.

Analogia: GPS e a UML

Um GPS mostra o mesmo trajeto de formas diferentes: vista de satélite, vista de ruas, vista de trânsito em tempo real. Nenhuma vista sozinha é “a errada” — cada uma serve pra uma decisão diferente. A UML funciona igual: Casos de Uso, Classes, Atividades e Sequência são “vistas” diferentes do mesmo sistema. Escolher o diagrama certo é escolher a vista certa pra pergunta que vocês têm agora.

3. Visão geral da Linguagem UML

UML (Unified Modeling Language) é a linguagem padrão para modelar sistemas orientados a objetos. Ela reúne vários tipos de diagrama, cada um mostrando uma visão diferente do mesmo sistema.

3.1 As duas grandes famílias de diagramas

A UML 2.x define ao todo 14 tipos de diagrama, divididos em duas famílias — mas esta disciplina foca nos principais, que cobrem a grande maioria das necessidades do dia a dia:

- Estruturais — mostram o que existe: Classes, Objetos, Componentes, Implantação, Pacotes, Estrutura Composta, Perfil.
- Comportamentais — mostram o que acontece: Casos de Uso, Atividades, Sequência, Máquina de Estados, Comunicação, Tempo, Visão Geral de Interação.

Os diagramas que vamos desenhar ao longo da disciplina estão destacados abaixo:

Diagrama	Tipo	Quando veremos
Casos de Uso	Comportamental	Aula 4
Classes	Estrutural	Aulas 5 e 6
Atividades	Comportamental	Aula 7
Sequência	Comportamental	Aula 8
Objetos, Componentes, Implantação, Estados	Estrutural / Comportamental	Visão panorâmica hoje

3.2 O que cada diagrama mostra

Diagrama de Casos de Uso (comportamental)

Mostra quem interage com o sistema (os atores, representados por bonecos) e o que cada um pode fazer (os casos de uso, representados por elipses), dentro de um retângulo que representa os limites do sistema. Exemplo: um ator “Cliente” ligado a um caso de uso “Fazer Pedido”. É o diagrama que vamos aprender a desenhar na Aula 4.

Diagrama de Classes (estrutural)

Mostra as “entidades” do sistema (as classes), seus atributos, seus métodos e como elas se relacionam entre si, com números indicando a multiplicidade da relação (ex.: um Pedido tem de 1 a N Itens). É o diagrama mais usado na prática profissional, e veremos ele nas Aulas 5 e 6.

Diagrama de Atividades (comportamental)

Mostra o passo a passo de um processo ou fluxo de trabalho, com caixas de ação conectadas por setas e losangos representando decisões (“se sim, siga por aqui; se não, siga por ali”) — parecido com um fluxograma. Veremos na Aula 7.

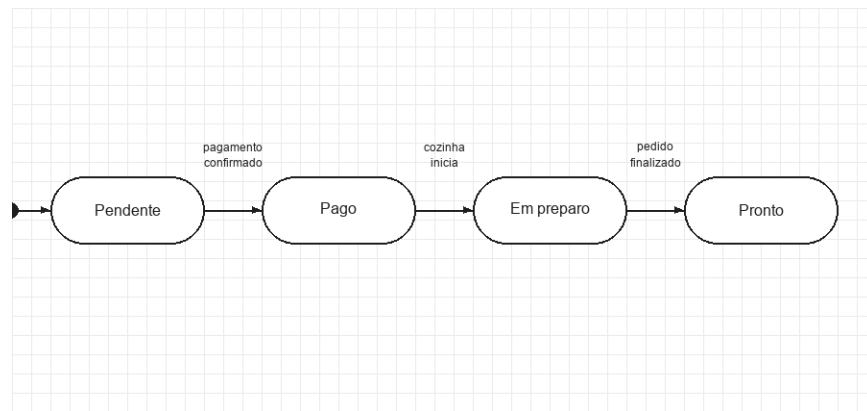
Diagrama de Sequência (comportamental)

Mostra a troca de mensagens entre objetos ao longo do tempo, em ordem: cada objeto tem uma “linha de vida” vertical, e setas horizontais numeradas representam as mensagens indo de um objeto para outro. Veremos na Aula 8.

3.2.1 Panorama: mais quatro diagramas (de reconhecimento)

Diagrama de Estados

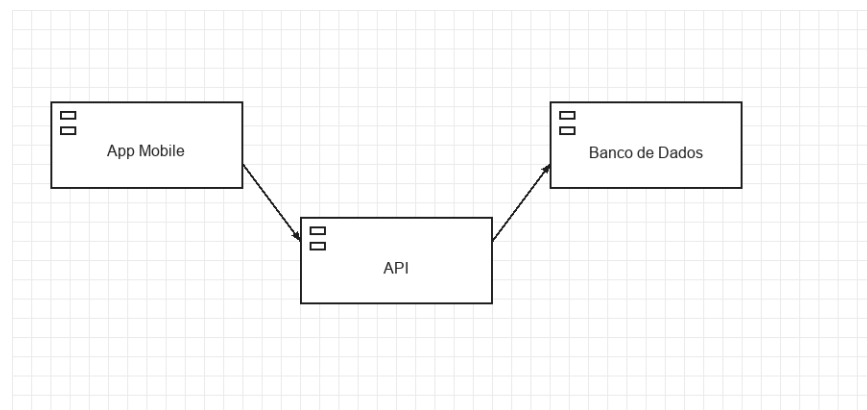
Mostra os estados possíveis de um objeto e o que dispara a mudança de um pro outro. Útil pra regras com “status”: pedido Pendente → Pago → Em preparo → Pronto. Não é foco desta disciplina, mas aparece bastante em sistemas de workflow.



Exemplo: Diagrama de Estados do Pedido do TechBurger.

Diagrama de Componentes

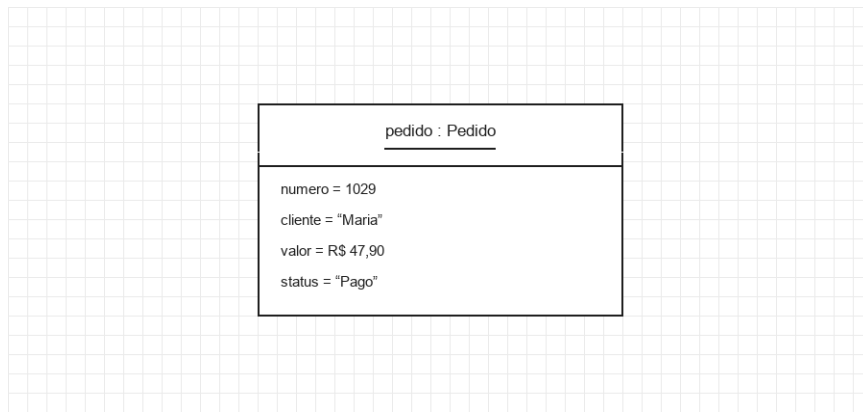
Mostra como o sistema se divide em partes que se comunicam (ex.: API, banco de dados, app mobile). Ajuda a enxergar a arquitetura de fora, sem entrar no código.



Exemplo: Diagrama de Componentes do TechBurger.

Diagrama de Objetos

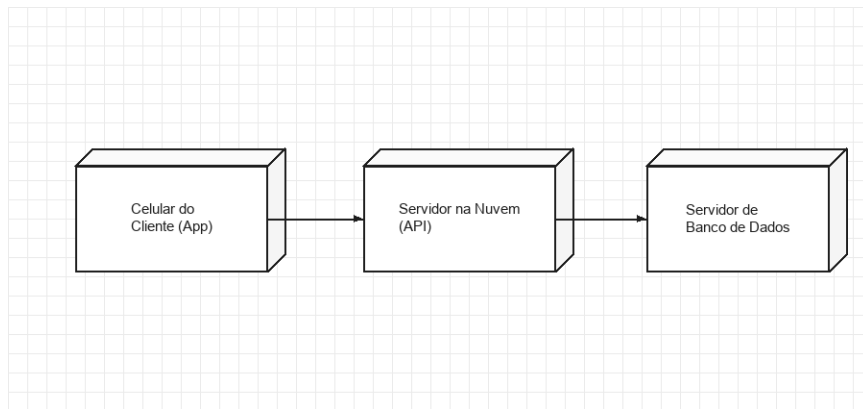
Mostra uma instância específica, já com valores reais preenchidos, num instante do tempo — é como uma “foto”: um Pedido nº 1029, cliente “Maria”, valor R\$ 47,90, não o modelo geral.



Exemplo: Diagrama de Objetos — uma instância real do Pedido.

Diagrama de Implantação

Mostra como as partes do sistema são distribuídas em servidores físicos ou na nuvem — ex.: app no celular, API na nuvem, banco de dados num servidor separado.



Exemplo: Diagrama de Implantação do TechBurger.

Notas de mercado

- Cada classe de um Diagrama de Classes costuma virar uma tabela no banco de dados, e cada atributo uma coluna — vocês vão aprofundar isso na disciplina de Banco de Dados (também no Módulo II).
- Um quadro Kanban (Trello, Jira) mostra a mesma ideia de um Diagrama de Atividades, mas sem decisões explícitas — o losango de decisão é o que o Kanban simples não tem.
- Um RNF como “o sistema deve confirmar o pagamento em até 2 segundos” vira, na prática, um tempo esperado entre duas setas de um Diagrama de Sequência (Aula 8).

3.3 Elementos comuns a quase todos os diagramas UML

- Ator — uma pessoa, sistema externo ou papel que interage com o que está sendo modelado.
- Elemento — a “caixa” ou símbolo que representa uma classe, um objeto, uma ação etc.
- Relacionamento — a linha que conecta dois elementos, mostrando como eles se associam.
- Rótulo (label) — o texto que nomeia um elemento ou explica um relacionamento.

No TechBurger: o Ator seria o Cliente ou o Atendente; um Elemento seria a classe Pedido ou o caso de uso “Fazer Pedido”; o Relacionamento é a linha que liga o Cliente ao caso de uso; e um Rótulo é, por exemplo, o texto “1..*” numa relação de Classes, ou “Sim”/“Não” numa seta de Atividades.

Mini-exercício: identifique o ator

Cenário 1: o Cliente faz login e escolhe os itens pelo app. Cenário 2: o sistema de pagamento externo avisa que a cobrança foi aprovada. Cenário 3: o Atendente cancela um pedido pelo painel da loja. Pra cada cenário: quem (ou o quê) é o ator?

Resposta: Cliente (pessoa) e Atendente (pessoa) são atores óbvios; o sistema de pagamento também é um ator, mesmo sendo outro sistema e não uma pessoa — regra prática: ator é quem interage de fora pra dentro do sistema, pessoa ou sistema externo, tanto faz.

3.4 Prévia da Aula 4: elementos do Diagrama de Casos de Uso

Vamos usar o Diagrama de Casos de Uso a fundo na Aula 4, mas aqui vai uma prévia dos elementos que o compõem.

- Ator: representa quem ou o que interage com o sistema (pessoa, outro sistema). Notação: boneco palito com o nome embaixo.
- Caso de uso: representa uma funcionalidade que o sistema oferece ao ator. Notação: elipse com o nome dentro (verbo no infinitivo + complemento, ex.: “Fazer Pedido”).
- Sistema (fronteira): um retângulo que delimita o que está dentro do sistema (os casos de uso) do que está fora (os atores).
- Regra prática: se está fora do retângulo e interage com o sistema, é ator; se está dentro e é uma funcionalidade, é caso de uso.

Hoje já vamos desenhar uma versão simples disso (seção 5.2, TechBurger) e treinar sozinhos (seção 5.5, Biblioteca) — o aprofundamento com Include/Extend fica pra Aula 4.

3.4.1 Atividade: Diagrama Humano

Antes de ir para o computador, vamos montar um Diagrama de Casos de Uso com o próprio corpo da turma — uma forma rápida de sentir a regra prática (ator fora, caso de uso dentro) antes de desenhar.

- O professor marca uma fronteira no chão (fita crepe, corda ou o corredor entre as fileiras) representando o retângulo do sistema.
- Voluntários viram Atores e ficam FORA da fronteira; outros viram Casos de Uso e ficam DENTRO, dizendo em voz alta a ação que representam (“Fazer Pedido”, “Pagar Pedido”, “Preparar Pedido”).
- A turma ajuda a “desenhar” as ligações (um barbante, ou só apontando) entre cada ator e o caso de uso que ele realiza.
- Rodada de erro proposital: o professor move alguém pro lado errado da fronteira e a turma aponta o erro, usando a regra prática.

Fechamento: o que a turma acabou de fazer com o corpo é exatamente o que o draw.io representa com bonecos e elipses — próximo passo, na tela.

3.5 Como escolher o diagrama certo

Pergunte primeiro: isso é uma COISA (estrutura) ou uma AÇÃO (comportamento)? Quer mostrar quem interage com o sistema? Casos de Uso. Quer mostrar os dados que uma entidade guarda? Classes. Quer mostrar a ordem dos passos de um processo? Atividades (o processo em si) ou Sequência (a troca de mensagens entre objetos/sistemas) — na dúvida entre os dois, lembre: Atividades é o processo, Sequência é a conversa.

Erro comum na escolha do diagrama

Cenário: “o app manda o pedido pro sistema de pagamento, que responde aprovando ou recusando”. Erro comum: desenhar isso como Diagrama de Atividades (processo interno). Por quê é erro: são dois sistemas diferentes trocando mensagens — isso é conversa entre objetos/sistemas, não um processo interno de um só. Escolha certa: Diagrama de Sequência (Aula 8).

3.6 Quiz relâmpago: qual diagrama usar?

Para cada frase abaixo, identifiquem qual dos 4 diagramas estudados (Casos de Uso, Classes, Atividades ou Sequência) se aplica.

1. 1. Mostrar quem pode fazer o quê no sistema.
2. 2. Mostrar os atributos e métodos de uma entidade.
3. 3. Mostrar o passo a passo de um processo, com decisões.
4. 4. Mostrar a ordem exata de mensagens entre dois sistemas.
5. 5. Mostrar que um Pedido pode ter vários Itens, e cada Item pertence a um Pedido.
6. 6. Mostrar o Cliente escolhendo o cardápio e fazendo o pedido no app.
7. 7. Mostrar como os componentes do sistema estão distribuídos em servidores diferentes.
8. 8. Mostrar um Pedido específico, já com valores reais preenchidos nos atributos.

4. Ferramentas de modelagem

Ferramenta	Características
draw.io / diagrams.net	Gratuita, roda no navegador, boa para começar. Usada nesta disciplina.
StarUML / Astah	Ferramentas dedicadas a UML, com validação de notação.
Lucidchart	Colaborativa, baseada em nuvem.
PlantUML	Cria diagramas a partir de texto (código), ótimo para versionar junto com o projeto em um repositório Git.
Visual Paradigm / Enterprise Architect	Ferramentas profissionais completas, usadas em empresas maiores, com recursos de geração de código.

4.1 Por que usar uma ferramenta em vez de desenhar à mão?

- Versionamento — dá para guardar o histórico de mudanças do diagrama, igual ao código.
- Padronização — a ferramenta obriga a usar a notação UML correta, evitando símbolos inventados.
- Reuso — elementos (classes, atores) podem ser reaproveitados entre diagramas do mesmo projeto.
- Exportação — o diagrama pode virar imagem, PDF ou até esqueleto de código, dependendo da ferramenta.

Quando usar qual ferramenta

- Quer algo rápido e gratuito, sem instalar nada → draw.io.

- Precisa que a notação UML esteja sempre 100% correta → StarUML / Astah.
- O time inteiro precisa editar o mesmo diagrama ao vivo → Lucidchart.
- Quer versionar o diagrama como código, junto com o projeto → PlantUML.
- Nesta disciplina usamos draw.io — mas a lógica de escolha vale pro mercado também.

4.2 Caça ao tesouro no draw.io

- Na busca de formas do draw.io, encontrem: 1 ator, 1 caso de uso, 1 classe (com compartimentos) e 1 raia (swimlane).
- Arrastem cada um pro canvas e dêem um nome qualquer. Tempo: 5 minutos.
- Terminou antes? Ache também o símbolo de «extend» (ponto de extensão).

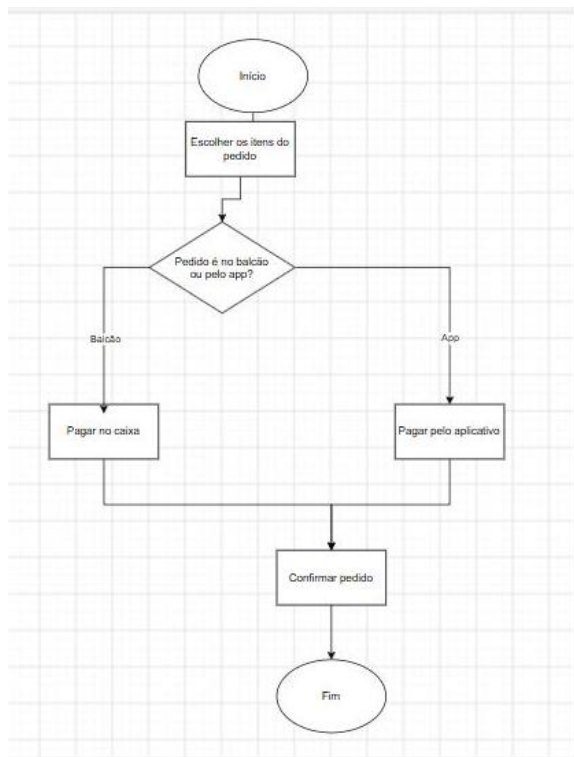
draw.io: atalhos úteis

- Ctrl+Shift+H — ajusta o zoom pra ver o diagrama inteiro na tela.
- Ctrl+D — duplica a forma selecionada (útil pra repetir várias elipses de Casos de Uso).
- Ctrl+Shift+V — cola só o estilo de uma forma em outra.
- Segurar Shift ao arrastar uma seta — mantém a linha reta.

5. Atividade prática

5.1 Prática guiada no draw.io

Acompanhe o professor reproduzindo o fluxograma de pedido do TechBurger no draw.io (passo a passo nos slides da aula) e salve seu próprio arquivo de teste.



Resultado esperado: fluxograma do pedido do TechBurger.

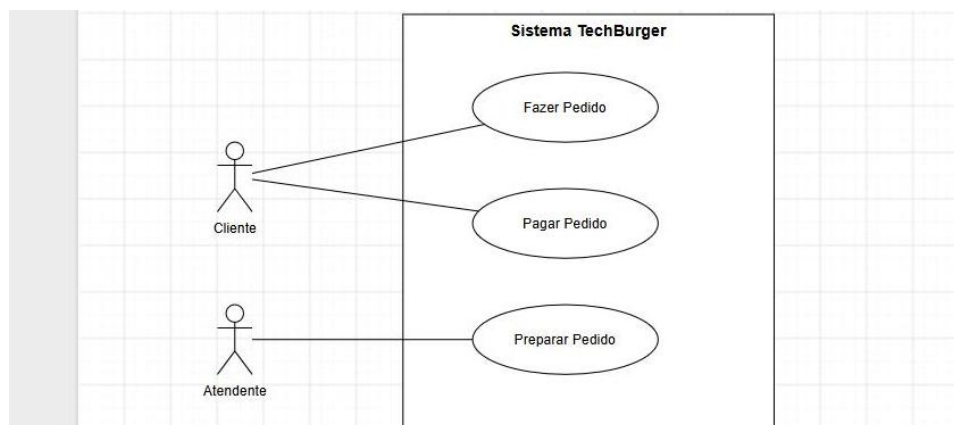
Erros comuns no draw.io

- Forma “flutuando”: parece conectada, mas a seta só está por cima, sem virar vértice de verdade — mova a forma pra testar.
- Esquecer de salvar (Ctrl+S) antes de fechar a aba.
- Nomear caso de uso sem verbo no infinitivo (“Pedido” em vez de “Fazer Pedido”).
- Dica de conferência: arrastem uma forma conectada — se a seta se mover junto, está conectada de verdade.

5.2 Prática guiada no draw.io: esboço de Casos de Uso

Acompanhe o professor reproduzindo, no draw.io, o esboço de Casos de Uso do TechBurger — o mesmo cenário do fluxograma, agora sob o olhar de quem interage com o sistema.

- Arrastem um retângulo grande e nomeiem “Sistema TechBurger” — ele é a fronteira do sistema.
- Arrastem a forma de ator (boneco palito) para FORA do retângulo: um para Cliente, outro para Atendente.
- Arrastem elipses para DENTRO do retângulo e nomeiem com um verbo no infinitivo: “Fazer Pedido”, “Pagar Pedido”, “Preparar Pedido”.
- Conectem cada ator à elipse que ele realiza, com uma linha simples — sem seta, sem texto. Cliente faz e paga o pedido; Atendente prepara o pedido.
- Regra prática: atores sempre fora do retângulo; casos de uso sempre dentro; ligações sem seta.



Esboço completo: Diagrama de Casos de Uso do TechBurger.

5.3 Classificando diagramas

Em grupo, leiam as descrições abaixo e classifiquem cada diagrama como Estrutural ou Comportamental, indicando também o tipo de diagrama UML correspondente.

#	Descrição	Tipo (Estrutural/Comportamental)	Nome do diagrama
1	Um diagrama mostra as classes Cliente, Pedido e Produto conectadas por linhas com números (multiplicidade) indicando quantas instâncias se relacionam.		
2	Um diagrama mostra bonecos (atores) fora de um retângulo com o nome do sistema, e elipses dentro representando ações como “Fazer Pedido”.		

3	Um diagrama mostra caixas conectadas por setas representando um fluxo de trabalho, com losangos indicando decisões.		
4	Um diagrama mostra linhas verticais tracejadas (linhas de vida) e setas horizontais numeradas trocando mensagens entre objetos.		
5	Um diagrama mostra módulos de software conectados por linhas indicando como serão distribuídos em servidores físicos.		
6	Um diagrama mostra um objeto específico já com valores preenchidos em seus atributos, representando um instante no tempo.		
7	Um diagrama mostra um Pedido no estado “Pendente” virando “Pago” quando o pagamento é confirmado.		
8	Um diagrama mostra o app mobile, a API e o banco de dados como blocos separados, indicando como o sistema se divide em partes.		

5.4 Agora sozinho: seu fluxograma

Sem o professor guiando, criem um segundo fluxograma para a regra da Biblioteca “Ler Mais” (Aulas 1-2): um associado só pode pegar um livro novo se tiver menos de 3 emprestados.

O fluxograma deve ter pelo menos 1 losango de decisão e 2 caixas de ação. Terminou antes? Acrescente o caso do professor (prazo de 30 dias em vez de 14).

5.5 Agora sozinho: seu esboço de Casos de Uso

Usando a regra da Biblioteca “Ler Mais” (mesma regra do fluxograma que vocês já fizeram): desenhem 1 ator (Aluno) e 2 casos de uso — “Pegar livro emprestado”, “Devolver livro”.

- Conectem o ator a cada caso de uso que ele realiza, dentro do retângulo “Sistema da Biblioteca”.
- Não se preocupem com relações mais avançadas (Include/Extend) — isso é conteúdo da Aula 4.
- Tempo: 5 minutos.

5.6 Desafio bônus: fluxograma do estoque

Terminaram os dois fluxogramas? Tentem mais um. RF do TechBurger: “se o item selecionado estiver sem estoque, o sistema sugere um substituto” (Regra 6, Aula 1).

Fluxograma esperado: Início → Selecionar item → Item em estoque? → Sim: Confirmar item / Não: Sugerir substituto → Fim. Sem tempo definido — é só para quem sobrar tempo.

6. Erros comuns ao modelar

6.1 Anti-padrões mais comuns

- Modelar demais: diagrama tão detalhado que ninguém lê, ou que vira mais trabalho que o próprio código.
- Modelar de menos: diagrama tão vago que não ajuda a tirar nenhuma dúvida real.

- Diagrama desatualizado: o sistema mudou, mas o diagrama continua mostrando a versão antiga — pior que não ter diagrama nenhum.
- Notação inventada: usar símbolos fora do padrão UML, que só quem desenhou entende.
- Modelar sem validar com quem usa: o diagrama fica “bonito” no papel, mas ninguém do time confirma se reflete a realidade.
- Copiar um modelo pronto da internet sem adaptar: a notação bate, mas as regras de negócio do seu sistema são outras.

Sinal de alerta: se dá para apagar o diagrama e nada muda no código, ele não estava sendo usado de verdade.

Erros comuns ao modelar: e no seu Projeto Integrador?

Nas Aulas 16 a 19 vocês vão modelar um sistema próprio, do zero, em grupo. Os mesmos erros desta seção valem lá: modelar demais, modelar de menos, diagrama desatualizado, notação inventada, não validar com o grupo. Comecem já a prestar atenção: qual desses erros o seu grupo tem mais tendência a cometer?

6.2 Encontre o erro

Um colega desenhou um Diagrama de Classes há 6 meses e nunca mais abriu o arquivo; o sistema ganhou 3 atributos novos desde então, mas o diagrama continua igual.

Em dupla: qual erro comum (seção 6.1) esse cenário representa? O que vocês fariam diferente daqui pra frente? Tempo: 5 minutos.

Mais um cenário: encontre o erro 2

Um grupo entregou um Diagrama de Classes gigante, com 30 classes e todos os atributos possíveis, “pra não faltar nada”. Ninguém do grupo consegue explicar o diagrama de cabeça — precisam consultar o arquivo pra responder qualquer pergunta sobre ele. Qual erro comum é esse? O que o grupo deveria ter feito diferente?

Glossário

Termo	Significado
Modelo	Representação simplificada de um sistema, mostrando o que importa para um propósito.
UML	Unified Modeling Language — linguagem padrão para modelar sistemas orientados a objetos.
Modelo estrutural	Representa o que existe no sistema (classes, objetos, componentes).
Modelo comportamental	Representa o que acontece no sistema (casos de uso, atividades, sequência).
Abstração	Mostrar apenas o que é relevante para o objetivo do modelo.
OMG (Object Management Group)	Organização internacional responsável por padronizar e manter a UML.
Ator	Pessoa, sistema externo ou papel que interage com o sistema sendo modelado.
Notação	Conjunto de símbolos e regras usados para representar elementos em um diagrama.
Custo de mudança	Quanto mais tarde um erro é encontrado no desenvolvimento, mais caro fica corrigi-lo.

Critério de Aceite	Condição objetiva (Dado/Quando/Então) que diz quando um Requisito Funcional está pronto e correto — visto na retomada da Aula 2, no início desta aula.
Gherkin	Formato de escrita de critérios de aceite no padrão Dado/Quando/Então, usado depois em testes de aceitação.

Perguntas de revisão

9. 1. O que é modelagem de sistemas e por que ela é feita antes (ou durante) o desenvolvimento?
10. 2. Diferencie modelo estrutural de modelo comportamental, dando um exemplo de cada.
11. 3. Cite duas características de um bom modelo.
12. 4. Qual ferramenta usaremos principalmente nesta disciplina para desenhar diagramas?
13. 5. Cite três diagramas UML que estudaremos nas próximas aulas e classifique cada um como estrutural ou comportamental.
14. 6. Por que um erro encontrado em produção custa muito mais para corrigir do que o mesmo erro encontrado na fase de requisitos?
15. 7. Quais três métodos foram unificados para dar origem à UML, e qual organização a padronizou?
16. 8. Releia a analogia da planta de um prédio (seção 2.4) e explique, com suas palavras, por que a UML tem vários diagramas em vez de um só.
17. 9. Qual diagrama UML você usaria para mostrar a troca de mensagens entre o aplicativo e o sistema de pagamento, em ordem cronológica? Justifique.
18. 10. Cite uma vantagem de desenhar diagramas em uma ferramenta (como draw.io) em vez de fazer à mão no papel.

Fechamento: seu checklist pessoal de modelagem

- Antes de desenhar: já sei se é uma COISA (estrutura) ou uma AÇÃO (comportamento)?
- Durante: meu modelo tem abstração, clareza, consistência e o nível de detalhe certo pro público?
- Depois: alguém que não sou eu consegue entender esse diagrama sem eu explicar?
- Sempre: esse diagrama ainda representa o sistema real, ou já ficou desatualizado?